
Reinforcement Learned Planning: Gradient Plan Optimization

Guided by a Learned Reward, Not Latent Geometry

Anonymous Author(s)

Affiliation

email

1 Introduction

Planning inside a learned world model (WM) is today dominated by *sampling*: the Cross-Entropy Method (CEM) and MPPI draw hundreds of action sequences, roll each through the WM, and iterate the sampling distribution toward the low-cost survivors. This zeroth-order search is robust—it needs only the *ranking* of sampled plans to be roughly right—but costs thousands of rollouts per decision and scales badly with plan dimension and horizon, exactly where the ranking signal thins out. A learned WM is differentiable, so the natural alternative is first-order: backpropagate a plan cost through the rollout and descend it in action space. This is rarely used—but not for the reason usually given. The failure is *not* that WM gradients are uninformative; it is *what people take the gradient of*. The standard cost is the WM’s *latent structure*: Euclidean distance between predicted-terminal and goal embeddings. For self-supervised encoders (DINO, LeJEPa, VICReg) this geometry is not a metric of *reachability*—nearby embeddings need not be few actions apart—so its gradient points in directions that do not reduce task cost. Descent stalls in false basins; CEM survives only because coarse ranking tolerates a mis-shaped landscape a gradient cannot.

Gradient planning must therefore be made into *its own algorithm*, with two changes. (i) **The objective must be a learned reward**: a goal-conditioned cost-to-go trained for temporal distance, not the encoder’s incidental latent geometry. (ii) **The optimizer must be learned**, not vanilla descent: a compact update rule, trained by backpropagation through the frozen WM, that takes the value’s gradient as a *feature* and learns how to move the plan. We call the result **Reinforcement Learned Planning** (RLP). It plans with ~ 16 rollout-equivalents per step (vs. $\sim 9,000$ for CEM), deterministically, and—across two benchmarks and four world models—beats the sampling planner most exactly where the WM’s native latent cost is weakest.

2 Method

Setup. A frozen WM W over latents z ; a plan is $\mathbf{A} = (A_1, \dots, A_H)$ of H action blocks. From an encoded history the WM unrolls autoregressively, differentiable in $\mathbf{A}$: $\hat{z}_t = W(\hat{z}_{t-3:t-1}, a_{t-3:t-1})$, giving the terminal latent $\hat{z}_H(\mathbf{A})$.

Critic — a learned reward (cost-to-go). $V_\phi(z, z_g) \geq 0$ is a goal-conditioned temporal distance (quasimetric head), trained *offline* on logged latents by n -step expectile TD with hindsight goals, $\gamma=1$, reached flag r , true distance δ :

$$y = r\delta + (1-r)(n_{\text{eff}} + V_{\bar{\phi}}(z_{t+n}, z_g)), \quad \mathcal{L}_{\text{TD}}(\phi) = \mathbb{E}[\omega_\tau(u) \text{Huber}_\beta(u)], \quad u = V_\phi(z_t, z_g) - \text{sg}(y), \quad (1)$$

with expectile weight $\omega_\tau(u) = (1-\tau)$ if $u > 0$ else τ , and $\bar{\phi}$ an EMA copy. Low $\tau \ll 0.5$ makes V optimistic toward the minimum (shortest-path semantics). This learned reward—not the WM’s raw latent distance—is what the planner descends.

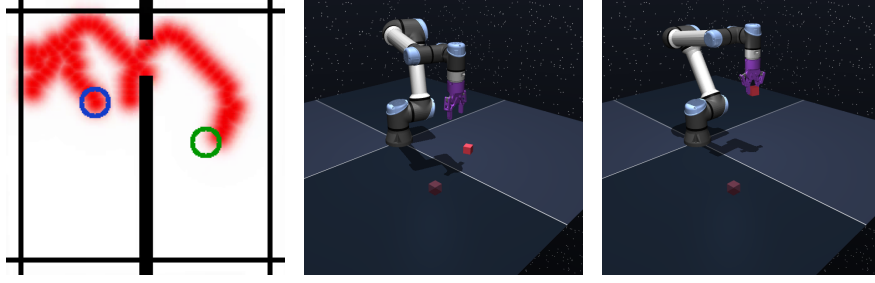


Figure 1: Benchmarks. *Left*: a full RLP trajectory on TwoRoom’s hard setting (h50, cross-wall goal): from the start (blue ring) the agent detours through the door gap to reach the goal room (goal = green ring); the red trail is the agent’s actual path, composited over the episode. *Middle, right*: OGBench cube — the arm must grasp the red block and place it within 4 cm of the goal pose (translucent cube), shown at the start and mid-transport. RLP plans in each WM’s frozen latent space from pixels/proprio only.

Actor — a learned optimizer over plans. f_θ is a weight-tied update rule applied K times from the zero plan:

$$\mathbf{A}^{(0)}=\mathbf{0}, \quad \mathbf{A}^{(k+1)} = \text{clip}(\mathbf{A}^{(k)} + f_\theta(\mathbf{A}^{(k)}, \widehat{\nabla_{\mathbf{A}} V}, E, k)), \quad E = V(\hat{z}_H(\mathbf{A}^{(k)}), z_g), \quad (2)$$

where $\widehat{\nabla_{\mathbf{A}} V}$ is the RMS-normalized gradient of the terminal value through the frozen WM. The goal enters *only* through V and its gradient—never as raw latent geometry—so the rule is reachability-guided by construction; no descent prior is hard-coded. θ is trained purely pathwise (no RL, no behavior cloning) by backpropagating the terminal value through the whole K -step chain and the frozen rollouts: $\mathcal{L}_{\text{actor}}(\theta) = \mathbb{E}[V(\hat{z}_H(\mathbf{A}^{(K)}), z_g) + \lambda \frac{1}{K} \sum_k V(\hat{z}_H(\mathbf{A}^{(k)}), z_g)]$. Critic and actor train in tandem with strict gradient separation ($\mathcal{L}_{\text{actor}}$ never updates ϕ); at deploy, K refinements from $\mathbf{A}^{(0)}=\mathbf{0}$, execute, replan—no sampling, no restarts.

3 Results

The baseline is the *no-move* (zero-action) policy—the agent does nothing—so success reflects only what *planning* contributes. All numbers are image-based planning, mean over 3 eval-seed task draws $\times$ 50 tasks, identical draws across planners; RLP uses the *same* learned-optimizer architecture on every WM, only the frozen WM changes. Tables 1 and 2 give the OGBench cube result two ways: floor-rescaled (the no-move floor is set to 0% and a perfect policy to 100%, so each entry is the fraction of above-floor headroom captured) and raw success with the floor as a base row. Horizon = goal offset in primitive steps; CEM = the WM’s native latent-distance cost searched by CEM ($\sim 9,000$ rollouts), RLP = ours (~ 16).

Table 1: OGBench cube, floor-rescaled (no-move = 0%, perfect = 100%): fraction of above-floor headroom captured. RLP dominates at both horizons, most on the weaker PLDM world model.

planner	LeWM (LeJEPa)		PLDM	
	h25	h75	h25	h75
no-move floor	0	0	0	0
Native CEM (latent)	55	41	35	26
RLP (ours)	81	78	57	43

Table 2: OGBench cube, raw success (%), with the no-move floor as a base row (block within 4 cm of goal).

planner	LeWM (LeJEPA)		PLDM	
	h25	h75	h25	h75
no-move floor	44.7	46.0	44.7	46.0
Native CEM (latent)	75.3	68.0	64.0	60.0
RLP (ours)	89.3	88.0	76.3	69.3

Table 3: TwoRoom navigation, raw success (%); std / hard = same-room / across-the-wall goals. LeWM row is the 3-seed result; PLDM the released pretrained base.

planner	std h25	std h50	hard h25	hard h50	overall
no-move floor	10.0	1.3	1.3	0.0	3.2
Latent+CEM	84.0	58.0	82.7	68.0	73.2
RLP (LeWM)	100.0	100.0	100.0	100.0	100.0
RLP (PLDM)	90.7	95.3	97.3	96.7	95.0

Efficiency (compute vs. wall-clock). RLP reaches a plan with ≈ 16 WM rollout-equivalents per decision (K refinement steps $\times$ 2 rollouts) versus CEM’s $\approx 9,000$ (300 samples $\times$ 30 iterations): a $\sim 560\times$ reduction in world-model compute, deterministically. Measured wall-clock (same env, same WM, h75) is ≈ 1.1 s per replan for RLP versus ≈ 9 s for CEM— $\sim 8\times$ faster. The wall-clock gap is far smaller than the compute gap only because CEM batches its 300 samples in parallel each iteration while *RLP is not parallelized yet*: its K refinement rollouts and per-environment plans run serially. Batching them closes most of that gap, so $\sim 8\times$ is a floor, not a ceiling.

Findings. (1) One learned-optimizer architecture transfers unchanged across four world models (two objectives $\times$ two benchmarks) and beats the sampling planner on every horizon. (2) RLP’s advantage is largest exactly where the WM’s latent cost is weakest (PLDM $<$ LeWM), and on LeWM-cube it holds with horizon as raw latent guidance decays but the learned reward does not. (3) It does so at $\sim 560\times$ less planning compute than CEM. Together: gradient planning works once it is given a learned reward and a learned optimizer instead of the encoder’s latent geometry and vanilla descent.

Notes. 0 = no-move (zero-action) policy. Cube LeWM h25 RLP = 89.3 (3-seed mean) on the negative-data (grasp-miss) fine-tuned LeWM with a negatives-augmented critic; on the authors’ stock checkpoint RLP = 88.0. The h75 column and all CEM baselines use the stock checkpoint. PLDM cube = 2-seed mean. TwoRoom LeWM = the LeWM-TwoRoom checkpoint, 100 on all 12 cells $\times$ 3 seeds (the authors’ released “lejepa” pretrained variant reaches 99.3); PLDM = released pretrained base, single seed. Latent+CEM is on the LeWM WM.